

LAPORAN PRATIKUM STRUKTUR DATA

“DOUBLE LINKED LIST”



OLEH:

KHAULA LATHIFA FIRDAUSYI

2511531007

MATA KULIAH STRUKTUR DATA

DOSEN PENGAMPU : Dr. WAHYUDI,
S.T, M.T

ASISTEN PRAKTIKUM : M. GALID A.

FAKULTAS TEKNOLOGI
INFORMASI DEPARTEMEN
INFORMATIKA
UNIVERSITAS ANDALAS
2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum Struktur Data ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum sekaligus sebagai bentuk penerapan materi yang telah dipelajari.

Dalam penyusunan laporan ini, penulis memperoleh banyak pengetahuan dan pengalaman, khususnya dalam memahami konsep struktur data Double Linked List serta penerapannya dalam pemrograman Java. Praktikum ini memberikan gambaran mengenai bagaimana teori yang telah dipelajari dapat diimplementasikan secara langsung dalam bentuk program sederhana.

Penulis menyadari bahwa laporan ini masih belum sempurna. Oleh karena itu, penulis mengharapkan saran yang dapat membantu dalam penyempurnaan laporan di masa mendatang.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	i
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Teori.....	2
2.2 Penjelasan Kode Program.....	2
2.2.1 Class NodeDLL_2511531007	2
2.2.2 Class InsertSLL_2511531007	3
2.2.3 Class PenelusuranDLL_2511531007	5
2.2.4 Class HapusDLL_2511531007.....	6
2.3 Tugas Praktikum	9
2.3.1 Class Lagu_2511531007	9
2.3.2 Class Musik_2511531007	10
BAB III PENUTUP	14
3.1 Kesimpulan	14
3.2 Saran	14
DAFTAR PUSTAKA.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan cara yang digunakan untuk menyimpan dan mengelola data agar dapat diproses dengan lebih efisien. Salah satu struktur data linear yang sering digunakan adalah linked list. Linked list terdiri dari kumpulan node yang saling terhubung, di mana setiap node menyimpan data dan pointer sebagai penghubung antar node.

Doubly Linked List merupakan jenis linked list yang memiliki dua pointer pada setiap node, yaitu `next` untuk menunjuk node berikutnya dan `prev` untuk menunjuk node sebelumnya. Dengan dua pointer tersebut, data dapat ditelusuri secara maju maupun mundur serta mempermudah proses penambahan dan penghapusan node.

Pada praktikum ini dilakukan implementasi doubly linked list menggunakan bahasa pemrograman Java. Program yang dibuat mencakup operasi dasar pada doubly linked list, seperti penambahan node, penghapusan node, dan penelusuran data. Melalui praktikum ini diharapkan dapat memahami konsep serta cara kerja doubly linked list dalam pengolahan data.

1.2 Tujuan

Tujuan dari pelaksanaan praktikum ini antara lain sebagai berikut:

1. Memahami konsep dan cara kerja struktur data doubly linked list.
2. Mengimplementasikan operasi dasar doubly linked list seperti penambahan, penghapusan, dan penelusuran node menggunakan bahasa pemrograman Java.

1.3 Manfaat

Manfaat dari praktikum ini antara lain sebagai berikut:

1. Memahami penggunaan pointer `next` dan `prev` pada doubly linked list.
2. Meningkatkan kemampuan dalam membuat program struktur data menggunakan Java.

BAB II PEMBAHASAN

2.1 Dasar Teori

Doubly Linked List merupakan salah satu struktur data linear yang terdiri dari kumpulan node yang saling terhubung. Setiap node pada doubly linked list memiliki tiga bagian, yaitu data, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev). Dengan adanya dua pointer tersebut, proses penelusuran data dapat dilakukan ke dua arah, yaitu maju dan mundur.

Pada doubly linked list, data disimpan secara dinamis sehingga ukuran penyimpanan dapat bertambah atau berkurang sesuai kebutuhan. Struktur data ini berbeda dengan array yang memiliki ukuran tetap. Selain itu, proses penambahan dan penghapusan node pada posisi tertentu menjadi lebih mudah karena setiap node mengetahui node sebelum dan sesudahnya.

Beberapa operasi dasar pada doubly linked list meliputi insertion, deletion, dan traversal. Insertion digunakan untuk menambahkan node baru ke dalam linked list, baik di awal, akhir, maupun posisi tertentu. Deletion digunakan untuk menghapus node dari linked list, sedangkan traversal digunakan untuk menelusuri atau menampilkan data yang terdapat pada linked list.

Dalam bahasa pemrograman Java, doubly linked list dapat diimplementasikan menggunakan class node yang berisi data dan pointer. Setiap node dihubungkan menggunakan referensi object sehingga membentuk hubungan antar node dalam satu linked list. Praktikum implementasi doubly linked list membantu memahami konsep hubungan data dan penggunaan pointer dalam struktur data linear.

2.2 Penjelasan Kode Program

2.2.1 Class NodeDLL_2511531007

```
1 package pekan6_2511531007;
2
3 public class NodeDLL_2511531007 {
4     // mendefinisikan kelas Node
5     int data_1007; // data
6     NodeDLL_2511531007 next_1007; // Pointer ke next node
7     NodeDLL_2511531007 prev_1007; // Pointer ke previous node
8
9     // konstruktor
10    public NodeDLL_2511531007(int data_1007) {
11        this.data_1007 = data_1007;
12        this.next_1007 = null;
13        this.prev_1007 = null;
14    }
15 }
16
```

Kode program di atas digunakan untuk membuat satu node (simpul) pada struktur data doubly linked list. Berbeda dengan singly linked list, node pada doubly linked list memiliki dua arah penghubung, yaitu ke node berikutnya dan ke node sebelumnya.

Di dalam class ini terdapat tiga komponen utama. Yang pertama data_1007, berfungsi untuk menyimpan nilai data pada node dengan tipe data int. Yang kedua next_1007, yaitu pointer atau penunjuk ke node berikutnya. Sedangkan yang ketiga prev_1007, digunakan sebagai pointer ke node sebelumnya. Dengan adanya dua pointer tersebut, setiap node dapat terhubung ke dua arah.

Kemudian terdapat constructor NodeDLL_2511531007(int data_1007). Constructor ini berfungsi untuk membuat node baru sekaligus mengisi nilai datanya saat objek dibuat. Nilai data_1007 diisi sesuai parameter yang diberikan. Setelah itu next_1007 dan prev_1007 di-set menjadi null, yang berarti node tersebut belum terhubung dengan node lain, baik node sebelum maupun sesudahnya.

2.2.2 Class InsertSLL_2511531007

```
1 package pekand_2511531007;
2
3 public class InsertDLL_2511531007 {
4     // Menambahkan node di awal DLL
5     static NodeDLL_2511531007 insertBegin_1007(NodeDLL_2511531007 head_1007, int data_1007) {
6         // Buat node baru
7         NodeDLL_2511531007 new_node_1007 = new NodeDLL_2511531007(data_1007);
8         // Jadikan pointer ke awal head
9         new_node_1007.next_1007 = head_1007;
10        // Jadikan pointer prev head ke new_node
11        if (head_1007 != null) {
12            head_1007.prev_1007 = new_node_1007;
13        }
14        return new_node_1007;
15    }
16
17    // Fungsi menambahkan node di akhir
18    public static NodeDLL_2511531007 insertEnd_1007(NodeDLL_2511531007 head_1007, int newData_1007) {
19        // Buat node baru
20        NodeDLL_2511531007 newNode_1007 = new NodeDLL_2511531007(newData_1007);
21        // Jika dll null jadikan head
22        if (head_1007 == null) {
23            head_1007 = newNode_1007;
24        }
25        else {
26            NodeDLL_2511531007 curr_1007 = head_1007;
27            while (curr_1007.next_1007 != null) {
28                curr_1007 = curr_1007.next_1007;
29            }
30            curr_1007.next_1007 = newNode_1007;
31            newNode_1007.prev_1007 = curr_1007;
32        }
33        return head_1007;
34    }
35
36    // Fungsi menambahkan node di posisi tertentu
37    public static NodeDLL_2511531007 insertAtPosition_1007(NodeDLL_2511531007 head_1007, int pos_1007, int new_data_1007) {
38        // Buat node baru
39        NodeDLL_2511531007 new_node_1007 = new NodeDLL_2511531007(new_data_1007);
40        if (pos_1007 == 1) {
41            new_node_1007.next_1007 = head_1007;
42            if (head_1007 != null) {
43                head_1007.prev_1007 = new_node_1007;
44            }
45            head_1007 = new_node_1007;
46            return head_1007;
47        }
48        NodeDLL_2511531007 curr_1007 = head_1007;
49        for (int i_1007 = 1; i_1007 < pos_1007 - 1 && curr_1007 != null; i_1007++) {
50            curr_1007 = curr_1007.next_1007;
51        }
52        if (curr_1007 == null) {
53            System.out.println("Posisi tidak ada");
54            return head_1007;
55        }
56        new_node_1007.prev_1007 = curr_1007;
57        new_node_1007.next_1007 = curr_1007.next_1007;
58        curr_1007.next_1007 = new_node_1007;
59        if (new_node_1007.next_1007 != null) {
60            new_node_1007.next_1007.prev_1007 = new_node_1007;
61        }
62        return head_1007;
63    }
64
65    public static void printList_1007(NodeDLL_2511531007 head_1007) {
66        NodeDLL_2511531007 curr_1007 = head_1007;
67        while (curr_1007 != null) {
68            System.out.print(curr_1007.data_1007 + " <-> ");
69            curr_1007 = curr_1007.next_1007;
70        }
71        System.out.println();
72    }
73
74    public static void main(String[] args) {
75        // Membuat dll <-> 3 <-> 5
76        NodeDLL_2511531007 head_1007 = new NodeDLL_2511531007(2);
77        head_1007.next_1007 = new NodeDLL_2511531007(3);
78        head_1007.next_1007.prev_1007 = head_1007;
79        head_1007.next_1007.next_1007 = new NodeDLL_2511531007(5);
80        head_1007.next_1007.prev_1007 = head_1007.next_1007;
81        // cetak DLL awal
82        System.out.print("DLL Awal: ");
83        printList_1007(head_1007);
84        // langkah 6 di akhir
85        System.out.print("simpul 6 ditambah di akhir: ");
86        int data_1007 = 6;
87        head_1007 = insertEnd_1007(head_1007, data_1007);
88        printList_1007(head_1007);
89        // Menambahkan node 4 di posisi 4
90        System.out.print("tambah node 4 di posisi 4: ");
91        int data2_1007 = 4;
92        int pos_1007 = 4;
93        head_1007 = insertAtPosition_1007(head_1007, pos_1007, data2_1007);
94        printList_1007(head_1007);
95    }
96 }
97
```

```
DLL Awal: 2 <-> 3 <-> 5 <->  
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->  
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->  
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->  
|
```

Kode program di atas digunakan untuk melakukan operasi penambahan node pada struktur data doubly linked list. Program ini memiliki beberapa fungsi, yaitu menambahkan node di awal list, di akhir list, dan pada posisi tertentu, lalu menampilkan isi linked list.

Pada method `insertBegin_1007`, program berfungsi untuk menambahkan node di bagian awal doubly linked list. Pertama dibuat node baru menggunakan constructor `NodeDLL_2511531007`. Setelah itu pointer `next_1007` dari node baru diarahkan ke head lama. Jika linked list sebelumnya tidak kosong, maka pointer `prev_1007` dari head lama diarahkan kembali ke node baru. Terakhir node baru dijadikan head dan dikembalikan sebagai head terbaru.

Method `insertEnd_1007` digunakan untuk menambahkan node di bagian akhir linked list. Program terlebih dahulu membuat node baru. Jika linked list masih kosong (`head_1007 == null`), maka node baru langsung dijadikan head. Namun jika linked list sudah memiliki isi, program akan melakukan traversal menggunakan variabel `curr_1007` sampai mencapai node terakhir. Setelah itu node baru dihubungkan ke akhir list dengan mengatur pointer `next_1007` dan `prev_1007`.

Method `insertAtPosition_1007` digunakan untuk menambahkan node pada posisi tertentu. Program pertama membuat node baru, lalu memeriksa apakah posisi yang diminta adalah posisi pertama. Jika iya, maka prosesnya sama seperti insert di awal. Jika bukan posisi pertama, program melakukan traversal hingga node sebelum posisi yang diinginkan. Setelah posisi ditemukan, pointer `prev_1007` dan `next_1007` diatur agar node baru terhubung dengan node sebelumnya dan node sesudahnya. Jika posisi tidak ditemukan, program akan menampilkan pesan "Posisi tidak ada".

Method `printList_1007` digunakan untuk menampilkan isi doubly linked list. Program melakukan traversal mulai dari head menggunakan variabel `curr_1007`, lalu mencetak data setiap node dengan tanda `<->` sampai mencapai node terakhir.

Pada method main, program pertama membuat doubly linked list awal dengan isi 2 <-> 3 <-> 5. Setelah itu program mencetak linked list awal menggunakan method printList_1007.

Selanjutnya program menambahkan node bernilai 1 di awal linked list menggunakan method insertBegin_1007, lalu menampilkan hasilnya. Setelah itu program menambahkan node bernilai 6 di akhir linked list menggunakan method insertEnd_1007, kemudian mencetak hasilnya kembali.

Terakhir program menambahkan node bernilai 4 pada posisi ke-4 menggunakan method insertAtPosition_1007. Setelah proses selesai, linked list dicetak kembali untuk menampilkan hasil akhir dari seluruh operasi penambahan node.

2.2.3 Class PenelusuranDLL_2511531007

```
1 package pekan6_2511531007;
2
3 public class PenelusuranDLL_2511531007 {
4     // fungsi penelusuran maju
5     static void forwardTraversal_1007(NodeDLL_2511531007 head_1007) {
6         // memulai penelusuran dari head
7         NodeDLL_2511531007 curr_1007 = head_1007;
8         // lanjutkan sampai akhir
9         while (curr_1007 != null) {
10            // print data
11            System.out.print(curr_1007.data_1007 + " <-> ");
12            // pindah ke node berikutnya
13            curr_1007 = curr_1007.next_1007;
14        }
15        // print spasi
16        System.out.println();
17    }
18    // fungsi penelusuran mundur
19    static void backwardTraversal_1007(NodeDLL_2511531007 tail_1007) {
20        // mulai dari akhir
21        NodeDLL_2511531007 curr_1007 = tail_1007;
22        // lanjut sampai head
23        while (curr_1007 != null) {
24            // cetak data
25            System.out.print(curr_1007.data_1007 + " <-> ");
26            // pindah ke node sebelumnya
27            curr_1007 = curr_1007.prev_1007;
28        }
29        // cetak spasi
30        System.out.println();
31    }
32    public static void main(String[] args) {
33        // cetak DLL
34        NodeDLL_2511531007 head_1007 = new NodeDLL_2511531007(1);
35        NodeDLL_2511531007 second_1007 = new NodeDLL_2511531007(2);
36        NodeDLL_2511531007 third_1007 = new NodeDLL_2511531007(3);
37
38        head_1007.next_1007 = second_1007;
39        second_1007.prev_1007 = head_1007;
40        second_1007.next_1007 = third_1007;
41        third_1007.prev_1007 = second_1007;
42
43        System.out.println("Penelusuran maju:");
44        forwardTraversal_1007(head_1007);
45
46        System.out.println("Penelusuran mundur:");
47        backwardTraversal_1007(third_1007);
48    }
49 }
```

```
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

Kode program di atas digunakan untuk melakukan penelusuran atau traversal pada struktur data doubly linked list, baik secara maju maupun mundur. Program ini

memanfaatkan dua pointer pada setiap node, yaitu next_1007 dan prev_1007, sehingga data dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan.

Pada method forwardTraversal_1007, program digunakan untuk melakukan penelusuran maju mulai dari head. Variabel curr_1007 pertama kali diarahkan ke head linked list. Setelah itu program melakukan perulangan selama curr_1007 tidak bernilai null. Di dalam perulangan, data pada node dicetak menggunakan System.out.print(), lalu pointer dipindahkan ke node berikutnya menggunakan curr_1007 = curr_1007.next_1007. Proses ini terus berjalan sampai mencapai akhir linked list.

Method backwardTraversal_1007 digunakan untuk melakukan penelusuran mundur mulai dari tail atau node terakhir. Variabel curr_1007 pertama kali diarahkan ke tail linked list. Selanjutnya program melakukan perulangan selama curr_1007 tidak bernilai null. Pada setiap perulangan, data node dicetak, kemudian pointer dipindahkan ke node sebelumnya menggunakan curr_1007 = curr_1007.prev_1007. Proses berlangsung hingga mencapai node pertama.

Pada method main, program terlebih dahulu membuat tiga node dengan nilai 1, 2, dan 3. Setelah itu setiap node dihubungkan menggunakan pointer next_1007 dan prev_1007 sehingga membentuk doubly linked list 1 <-> 2 <-> 3.

Kemudian program menampilkan hasil penelusuran maju dengan memanggil method forwardTraversal_1007(head_1007). Hasil yang ditampilkan dimulai dari node pertama hingga node terakhir.

Setelah itu program menampilkan hasil penelusuran mundur dengan memanggil method backwardTraversal_1007(third_1007). Karena parameter yang diberikan adalah node terakhir, maka data akan ditampilkan dari belakang ke depan.

2.2.4 Class HapusDLL_2511531007

```
1 package pekan6_2511531007;
2
3 public class HapusDLL_2511531007 {
4     // fungsi menghapus node awal
5     public static NodeDLL_2511531007 delHead_1007(NodeDLL_2511531007 head_1007) {
6         if (head_1007 == null) {
7             return null;
8         }
9         NodeDLL_2511531007 temp_1007 = head_1007;
10        head_1007 = head_1007.next_1007;
11        if (head_1007 != null) {
12            head_1007.prev_1007 = null;
13        }
14        return head_1007;
15    }
16    // fungsi menghapus di akhir
17    public static NodeDLL_2511531007 delLast_1007(NodeDLL_2511531007 head_1007) {
18        if (head_1007 == null) {
19            return null;
20        }
21        if (head_1007.next_1007 == null) {
22            return null;
23        }
24        NodeDLL_2511531007 curr_1007 = head_1007;
25        while (curr_1007.next_1007 != null) {
26            curr_1007 = curr_1007.next_1007;
27        }
28        // update pointer previous node
29        if (curr_1007.prev_1007 != null) {
30            curr_1007.prev_1007.next_1007 = null;
31        }
32        return head_1007;
33    }
34 }
```

```

34 // fungsi menghapus node posisi tertentu
35 public static NodeDLL_2511531007 delPos_1007(NodeDLL_2511531007 head_1007, int pos_1007) {
36     // jika DLL kosong
37     if (head_1007 == null) {
38         return head_1007;
39     }
40     NodeDLL_2511531007 curr_1007 = head_1007;
41     // kelusuri sampai ke node yang akan dihapus
42     for (int i_1007 = 1; curr_1007 != null && i_1007 < pos_1007; i_1007++) {
43         curr_1007 = curr_1007.next_1007;
44     }
45     // jika posisi tidak ditemukan
46     if (curr_1007 == null) {
47         return head_1007;
48     }
49     // update pointer
50     if (curr_1007.prev_1007 != null) {
51         curr_1007.prev_1007.next_1007 = curr_1007.next_1007;
52     }
53     if (curr_1007.next_1007 != null) {
54         curr_1007.next_1007.prev_1007 = curr_1007.prev_1007;
55     }
56     // jika yang ingin dihapus head
57     if (head_1007 == curr_1007) {
58         head_1007 = curr_1007.next_1007;
59     }
60     return head_1007;
61 }
62 // fungsi mencetak DLL
63 public static void printlist_1007(NodeDLL_2511531007 head_1007) {
64     NodeDLL_2511531007 curr_1007 = head_1007;
65     while (curr_1007 != null) {
66         System.out.print(curr_1007.data_1007 + " ");
67         curr_1007 = curr_1007.next_1007;
68     }
69     System.out.println();
70 }
71 public static void main(String[] args) {
72     // buat sebuah DLL
73     NodeDLL_2511531007 head_1007 = new NodeDLL_2511531007(1);
74     head_1007.next_1007 = new NodeDLL_2511531007(2);
75     head_1007.next_1007.prev_1007 = head_1007;
76     head_1007.next_1007.next_1007 = new NodeDLL_2511531007(3);
77     head_1007.next_1007.next_1007.prev_1007 = head_1007.next_1007;
78     head_1007.next_1007.next_1007.next_1007 = new NodeDLL_2511531007(4);
79     head_1007.next_1007.next_1007.next_1007.prev_1007 = head_1007.next_1007.next_1007;
80     head_1007.next_1007.next_1007.next_1007.next_1007 = new NodeDLL_2511531007(5);
81     head_1007.next_1007.next_1007.next_1007.next_1007.prev_1007 = head_1007.next_1007.next_1007.next_1007;
82
83     System.out.print("DLL Awal: ");
84     printlist_1007(head_1007);
85
86     System.out.print("Setelah head dihapus: ");
87     head_1007 = delHead_1007(head_1007);
88     printlist_1007(head_1007);
89
90     System.out.print("Setelah node terakhir dihapus: ");
91     head_1007 = delLast_1007(head_1007);
92     printlist_1007(head_1007);
93
94     System.out.print("menghapus node ke 2: ");
95     head_1007 = delPos_1007(head_1007, 2);
96     printlist_1007(head_1007);
97 }
98
99 }

```

```

DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: 2 3 4
menghapus node ke 2: 2 4

```

Kode program di atas digunakan untuk melakukan operasi penghapusan node pada struktur data doubly linked list. Program ini memiliki beberapa fungsi, yaitu menghapus node di awal, di akhir, dan pada posisi tertentu, serta menampilkan isi linked list setelah dilakukan penghapusan.

Pada method delHead_1007, program digunakan untuk menghapus node pertama atau head dari doubly linked list. Pertama program memeriksa apakah linked list kosong dengan kondisi head_1007 == null. Jika kosong, method langsung mengembalikan null. Jika tidak kosong, head dipindahkan ke node berikutnya menggunakan head_1007 = head_1007.next_1007. Setelah itu pointer prev_1007 dari head baru diubah menjadi null agar tidak lagi terhubung dengan node lama yang sudah dihapus.

Method delLast_1007 digunakan untuk menghapus node terakhir pada linked list. Program terlebih dahulu memeriksa apakah linked list kosong atau hanya memiliki satu node. Jika hanya ada satu node, maka linked list akan menjadi kosong sehingga method mengembalikan null. Jika node lebih dari satu, program melakukan traversal

menggunakan variabel `curr_1007` sampai mencapai node terakhir. Setelah itu pointer `next_1007` dari node sebelumnya diubah menjadi null agar hubungan dengan node terakhir terputus.

Method `delPos_1007` digunakan untuk menghapus node pada posisi tertentu. Program pertama memeriksa apakah linked list kosong. Jika tidak kosong, program melakukan traversal hingga mencapai posisi node yang akan dihapus menggunakan perulangan `for`. Jika posisi tidak ditemukan, method langsung mengembalikan linked list tanpa perubahan. Setelah node ditemukan, pointer `next_1007` dari node sebelumnya dan pointer `prev_1007` dari node sesudahnya diperbarui agar node yang dihapus tidak lagi terhubung dalam linked list. Jika node yang dihapus adalah head, maka head dipindahkan ke node berikutnya.

Method `printList_1007` digunakan untuk mencetak isi doubly linked list. Program melakukan traversal mulai dari head menggunakan variabel `curr_1007`, lalu mencetak data setiap node sampai mencapai node terakhir.

Pada method `main`, program pertama membuat doubly linked list dengan isi 1 2 3 4 5. Setiap node dihubungkan menggunakan pointer `next_1007` dan `prev_1007` agar membentuk doubly linked list yang saling terhubung dua arah.

Selanjutnya program mencetak linked list awal menggunakan method `printList_1007`. Setelah itu program menghapus node pertama menggunakan method `delHead_1007`, lalu menampilkan hasilnya.

Kemudian program menghapus node terakhir menggunakan method `delLast_1007` dan kembali mencetak isi linked list setelah penghapusan dilakukan.

Terakhir program menghapus node pada posisi ke-2 menggunakan method `delPos_1007`. Setelah proses selesai, linked list dicetak kembali untuk menampilkan hasil akhir setelah seluruh operasi penghapusan dilakukan.

2.3 Tugas Praktikum

2.3.1 Class Lagu_2511531007

```
1 package tugasPraktikum_pekan6_2511531007;
2
3 public class Lagu_2511531007 {
4     // mendefinisikan kelas node
5     String judul_1007; // judul lagu
6     String penyanyi_1007; // penyanyi
7     Lagu_2511531007 next_1007; // pointer ke next node
8     Lagu_2511531007 prev_1007; // pointer ke previous node
9
10    // konstruktor
11    public Lagu_2511531007(String judul_1007, String penyanyi_1007) {
12        this.judul_1007 = judul_1007;
13        this.penyanyi_1007 = penyanyi_1007;
14        this.next_1007 = null;
15        this.prev_1007 = null;
16    }
17
18    // getter
19    public String getJudul_1007() { return judul_1007; }
20    public String getPenyanyi_1007() { return penyanyi_1007; }
21
22    // setter
23    public void setJudul_1007(String judul_1007) { this.judul_1007 = judul_1007; }
24    public void setPenyanyi_1007(String penyanyi_1007) { this.penyanyi_1007 = penyanyi_1007; }
25 }
```

Kode program di atas digunakan untuk membuat satu node pada struktur data doubly linked list dengan tema playlist lagu. Class Lagu_2511531007 berfungsi sebagai tempat penyimpanan data lagu sekaligus penghubung antar node dalam playlist.

Di dalam class ini terdapat empat atribut utama. Atribut judul_1007 digunakan untuk menyimpan judul lagu dengan tipe data String, sedangkan penyanyi_1007 digunakan untuk menyimpan nama penyanyi lagu. Selain itu terdapat next_1007 yang berfungsi sebagai pointer ke node berikutnya dan prev_1007 sebagai pointer ke node sebelumnya. Dengan adanya dua pointer tersebut, playlist dapat ditelusuri dari depan ke belakang maupun dari belakang ke depan.

Pada bagian constructor Lagu_2511531007(String judul_1007, String penyanyi_1007), constructor digunakan untuk membuat object lagu baru sekaligus mengisi data judul dan penyanyi saat object dibuat. Nilai judul_1007 dan penyanyi_1007 diisi sesuai parameter yang diberikan, sedangkan next_1007 dan prev_1007 diatur bernilai null karena node belum terhubung dengan node lain.

Program ini juga memiliki method getter dan setter. Method getter terdiri dari getJudul_1007() dan getPenyanyi_1007() yang digunakan untuk mengambil atau menampilkan nilai atribut judul dan penyanyi. Sedangkan method setter terdiri dari setJudul_1007() dan setPenyanyi_1007() yang digunakan untuk mengubah nilai atribut judul dan penyanyi pada object lagu.

2.3.2 Class Musik_2511531007

```
1 package tugasPraktikum_pekan_2511531007;
2
3 import java.util.Scanner;
4
5 public class Musik_2511531007 {
6     // ===== Sifat Sifat =====
7     public static Lagu_2511531007 tambahLagu_1007(Lagu_2511531007 head_1007, String judul_1007, String penyanyi_1007) {
8         // ===== Sifat Sifat =====
9         Lagu_2511531007 newLagu_1007 = new Lagu_2511531007(judul_1007, penyanyi_1007);
10        // ===== Sifat Sifat =====
11        if (head_1007 == null) {
12            head_1007 = newLagu_1007;
13        }
14        else {
15            Lagu_2511531007 curr_1007 = head_1007;
16            while (curr_1007.next_1007 != null) {
17                curr_1007 = curr_1007.next_1007;
18            }
19            curr_1007.next_1007 = newLagu_1007;
20            newLagu_1007.prev_1007 = curr_1007;
21        }
22        System.out.println("Lagu berhasil ditambahkan!");
23        return head_1007;
24    }
25    // ===== Sifat Sifat =====
26    public static Lagu_2511531007 hapusLagu_1007(Lagu_2511531007 head_1007) {
27        if (head_1007 == null) {
28            System.out.println("Playlist kosong!");
29            return null;
30        }
31        System.out.println("Lagu " + head_1007.judul_1007 + " berhasil dihapus");
32        head_1007 = head_1007.next_1007;
33        if (head_1007 != null) {
34            head_1007.prev_1007 = null;
35        }
36        return head_1007;
37    }
38    // ===== Sifat Sifat =====
39    static void tampilLagu_1007(Lagu_2511531007 head_1007) {
40        // ===== Sifat Sifat =====
41        Lagu_2511531007 curr_1007 = head_1007;
42        // ===== Sifat Sifat =====
43        System.out.println("===== Playlist Maju =====");
44        while (curr_1007 != null) {
45            // ===== Sifat Sifat =====
46            System.out.println(curr_1007.judul_1007 + " - " + curr_1007.penyanyi_1007);
47            // ===== Sifat Sifat =====
48            curr_1007 = curr_1007.next_1007;
49        }
50        // ===== Sifat Sifat =====
51        System.out.println();
52    }
53    // ===== Sifat Sifat =====
54    static void tampilLagu_1007(Lagu_2511531007 tail_1007) {
55        // ===== Sifat Sifat =====
56        Lagu_2511531007 curr_1007 = tail_1007;
57        // ===== Sifat Sifat =====
58        System.out.println("===== Playlist Mundur =====");
59        while (curr_1007 != null) {
60            // ===== Sifat Sifat =====
61            System.out.println(curr_1007.judul_1007 + " - " + curr_1007.penyanyi_1007);
62            // ===== Sifat Sifat =====
63            curr_1007 = curr_1007.prev_1007;
64        }
65        // ===== Sifat Sifat =====
66        System.out.println();
67    }
68    // ===== Sifat Sifat =====
69    static void cariLagu_1007(Lagu_2511531007 head_1007, String cariJudul_1007) {
70        if (head_1007 == null) {
71            System.out.println("Playlist kosong!");
72            return;
73        }
74        Lagu_2511531007 curr_1007 = head_1007;
75        boolean ditemukan_1007 = false;
76        while (curr_1007 != null) {
77            if (curr_1007.judul_1007.equalsIgnoreCase(cariJudul_1007)) {
78                System.out.println("Lagu ditemukan!");
79                System.out.println(curr_1007.judul_1007 + " - " + curr_1007.penyanyi_1007);
80                ditemukan_1007 = true;
81                break;
82            }
83            curr_1007 = curr_1007.next_1007;
84        }
85        if (!ditemukan_1007) {
86            System.out.println("Lagu tidak ditemukan!");
87        }
88    }
89    public static void main(String[] args) {
90        Scanner input_1007 = new Scanner(System.in);
91        Lagu_2511531007 head_1007 = null;
92        int pilihan_1007;
93        do {
94            System.out.println("===== Playlist Musik NIM: 2511531007 =====");
95            System.out.println("1. Tambah Lagu");
96            System.out.println("2. Hapus Lagu (Portama)");
97            System.out.println("3. Lihat Playlist (Maju)");
98            System.out.println("4. Lihat Playlist (Mundur)");
99            System.out.println("5. Cari Lagu");
100           System.out.println("6. Keluar");
101           System.out.println("Pilihan: ");
102           pilihan_1007 = input_1007.nextInt();
103           input_1007.nextLine();
104           switch (pilihan_1007) {
105               case 1:
106                   System.out.print("Judul Lagu: ");
107                   String judul_1007 = input_1007.nextLine();
108                   System.out.print("Penyanyi: ");
109                   String penyanyi_1007 = input_1007.nextLine();
110                   head_1007 = Musik_2511531007.tambahLagu_1007(head_1007, judul_1007, penyanyi_1007);
111                   break;
112               case 2:
113                   head_1007 = Musik_2511531007.hapusLagu_1007(head_1007);
114                   break;
115               case 3:
116                   Musik_2511531007.tampilMaju_1007(head_1007);
117                   break;
118               case 4:
119                   Musik_2511531007.tampilMundur_1007(tail_1007);
120                   break;
121               case 5:
122                   System.out.print("Masukkan judul lagu yang dicari: ");
123                   String cari_1007 = input_1007.nextLine();
124                   Musik_2511531007.cariLagu_1007(head_1007, cari_1007);
125                   break;
126               case 6:
127                   System.out.println("Program selesai");
128                   break;
129               default:
130                   System.out.println("Pilihan tidak tersedia!");
131               }
132        } while (pilihan_1007 != 6);
133        input_1007.close();
134    }
135 }
```

```

=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Risk It All
Penyanyi: Bruno Mars
Lagu berhasil ditambahkan!

=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 1
Judul Lagu: Ghost
Penyanyi: Justin Bieber
Lagu berhasil ditambahkan!
=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 5
Masukkan judul lagu yang dicari: ghost
Lagu ditemukan:
Ghost - Justin Bieber

=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 2
Lagu Risk It All berhasil dihapus

=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 6
Program selesai

```

```

=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 3

```

```

=== Playlist Maju ===
Risk It All - Bruno Mars
Ghost - Justin Bieber

```

```

=== Playlist Musik NIM: 2511531007 ===
1. Tambah Lagu
2. Hapus Lagu Pertama
3. Lihat Playlist (Maju)
4. Lihat Playlist (Mundur)
5. Cari Lagu
6. Keluar
Pilihan: 4

=== Playlist Mundur ===
Ghost - Justin Bieber
Risk It All - Bruno Mars

```

Kode program di atas digunakan untuk membuat program playlist musik sederhana menggunakan struktur data doubly linked list. Program ini memiliki beberapa fitur utama, yaitu menambahkan lagu, menghapus lagu pertama, menampilkan playlist secara maju dan mundur, serta mencari lagu berdasarkan judul.

Pada method `tambahLagu_1007()`, program digunakan untuk menambahkan lagu baru di akhir playlist. Pertama program membuat node lagu baru menggunakan constructor `Lagu_2511531007`. Jika playlist masih kosong (`head_1007 == null`), maka lagu baru langsung dijadikan head. Namun jika playlist sudah memiliki isi, program melakukan traversal menggunakan variabel `curr_1007` hingga mencapai node terakhir. Setelah itu node lagu baru dihubungkan ke akhir playlist dengan mengatur pointer `next_1007` dan `prev_1007`.

Method `hapusLaguAwal_1007()` digunakan untuk menghapus lagu pertama pada playlist. Program terlebih dahulu memeriksa apakah playlist kosong. Jika kosong,

program menampilkan pesan "Playlist kosong!". Jika tidak kosong, head dipindahkan ke node berikutnya menggunakan `head_1007 = head_1007.next_1007`. Kemudian pointer `prev_1007` pada head baru diubah menjadi null agar tidak lagi terhubung dengan node sebelumnya.

Method `tampilMaju_1007()` digunakan untuk menampilkan playlist dari awal ke akhir. Program memulai traversal dari head menggunakan variabel `curr_1007`. Selama `curr_1007` tidak bernilai null, data lagu berupa judul dan penyanyi dicetak ke layar, lalu pointer dipindahkan ke node berikutnya menggunakan `curr_1007 = curr_1007.next_1007`.

Method `tampilMundur_1007()` digunakan untuk menampilkan playlist dari akhir ke awal. Program menerima parameter berupa tail atau node terakhir playlist. Traversal dilakukan menggunakan pointer `prev_1007` sehingga lagu dapat ditampilkan dari belakang menuju depan.

Method `cariLagu_1007()` digunakan untuk mencari lagu berdasarkan judul. Program melakukan traversal mulai dari head menggunakan variabel `curr_1007`. Pencarian dilakukan menggunakan method `equalsIgnoreCase()` sehingga huruf besar dan kecil tidak memengaruhi hasil pencarian. Jika lagu ditemukan, program akan menampilkan data lagu tersebut. Jika tidak ditemukan, program menampilkan pesan "Lagu tidak ditemukan!".

Pada method `main`, program menggunakan class `Scanner` untuk menerima input dari pengguna. Variabel `head_1007` digunakan sebagai penunjuk node pertama pada playlist. Program kemudian menampilkan menu utama yang berisi beberapa pilihan operasi playlist.

Pada pilihan pertama, pengguna dapat menambahkan lagu dengan memasukkan judul dan penyanyi. Lagu kemudian ditambahkan ke akhir playlist menggunakan method `tambahLagu_1007()`.

Pada pilihan kedua, program menghapus lagu pertama menggunakan method `hapusLaguAwal_1007()`.

Pada pilihan ketiga, program menampilkan seluruh playlist dari awal ke akhir menggunakan method `tampilMaju_1007()`.

Pada pilihan keempat, program terlebih dahulu mencari node terakhir menggunakan variabel `tail_1007`, kemudian menampilkan playlist dari akhir ke awal menggunakan method `tampilMundur_1007()`.

Pada pilihan kelima, pengguna dapat mencari lagu berdasarkan judul menggunakan method `cariLagu_1007()`.

Sedangkan pada pilihan keenam, program akan berhenti dan menampilkan pesan "Program selesai". Program menggunakan perulangan `do while` sehingga menu akan terus muncul sampai pengguna memilih keluar dari program.

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data doubly linked list merupakan struktur data linear yang menggunakan dua pointer pada setiap node, yaitu next dan prev, sehingga data dapat ditelusuri secara maju maupun mundur. Implementasi doubly linked list dalam bahasa pemrograman Java dilakukan menggunakan class `NodeDLL_2511531007` sebagai pembentuk node yang saling terhubung.

Pada praktikum ini berhasil dilakukan beberapa operasi dasar doubly linked list, yaitu penambahan node pada awal, akhir, dan posisi tertentu menggunakan class `InsertDLL_2511531007`, penelusuran data secara maju dan mundur menggunakan class `PenelusuranDLL_2511531007`, serta penghapusan node pada awal, akhir, dan posisi tertentu menggunakan class `HapusDLL_2511531007`. Melalui praktikum ini, pemahaman mengenai cara kerja pointer dan manipulasi node pada doubly linked list menjadi lebih baik.

3.2 Saran

1. Mahasiswa diharapkan dapat lebih sering berlatih mengerjakan soal atau membuat program secara mandiri, sehingga saat menghadapi ujian akhir praktikum tidak mengalami kesulitan dalam memahami maupun menyelesaikan soal yang diberikan.
2. Selain itu, mahasiswa sebaiknya tidak hanya mengikuti langkah praktikum, tetapi juga mencoba memahami alur program agar dapat mengerjakan tanpa bergantung pada contoh yang sudah ada.
3. Dengan adanya latihan yang cukup, diharapkan proses pembelajaran praktikum dapat menjadi lebih efektif dan hasil yang diperoleh mahasiswa menjadi lebih maksimal.

DAFTAR PUSTAKA

- [1] W. Wahyudi, *Struktur Data dengan Java*. CV Eureka Media Aksara, 2025. [Online].
Available: <https://books.google.co.id/books?id=EN2lEQAAQBAJ>.